



NumPy

● PHASE 1: NUMPY BASICS

1) Introduction to NumPy (Why NumPy over Python lists)

1. Concept Explanation

What is NumPy?

NumPy (Numerical Python) is the foundational Python library for **numerical computing**. Its core feature is the `ndarray` (N-dimensional array), which is a compact, efficient structure for working with numeric data.

Intuition: what problem does NumPy solve?

In Data Science / AI-ML, most data can be represented as:

- **Vectors** (1D)
- **Matrices** (2D)
- **Tensors** (3D and higher)

Python lists can store numbers, but they are not optimized for large-scale numerical operations. NumPy is designed to make these operations:

- **fast** (performance)
- **memory efficient**
- **mathematically expressive** (clean, array-based syntax)

Why NumPy over Python lists? (the real reasons)

A) Data representation: Python objects vs raw numeric buffers

- A Python list stores **references** to Python objects.
 - Each number like `3` is a full Python object (with overhead).
- A NumPy array stores values in a **typed, contiguous block of memory**.
 - Example dtypes: `int32`, `int64`, `float32`, `float64`
 - Each element has a fixed number of bytes.

Why this matters:

- Lower overhead per element
- Better CPU cache usage
- Enables low-level optimizations

B) Speed: compiled loops + vectorization

With lists, you typically need Python loops for numeric operations.

NumPy enables **vectorization**, meaning:

- you express operations on whole arrays
- NumPy runs the heavy computation in optimized compiled code (C/Fortran)

Result:

- drastically faster operations on large data (typical in DS/ML)

C) True N-dimensional arrays with consistent rules

Nested lists can look like 2D data, but they are just “lists of lists” and do not provide:

- guaranteed rectangular shape
- fast multi-axis slicing
- shape-aware operations

NumPy arrays are true N-D arrays with consistent:

- **shape** rules
- **indexing** behavior
- **mathematical** behavior

D) Broadcasting (high-level motivation)

NumPy can often operate on arrays of different shapes by applying rules that conceptually “expand” smaller arrays without writing explicit loops.

Why this matters:

- shorter code
- fewer bugs
- performance (often avoids copies)

E) Standard language of the DS/ML ecosystem

NumPy is a common “base layer” for:

- Pandas (uses NumPy under the hood)
- SciPy
- scikit-learn
- Matplotlib

So learning NumPy makes other libraries much easier.

2. Syntax / Definition

Python list (general-purpose container)

A Python list is an ordered container that can hold **any Python objects**, including mixed types.

General idea:

- can hold numbers, strings, objects together
- numeric operations are not built-in (you usually loop)

NumPy `ndarray` (numerical, typed N-D array)

A NumPy `ndarray` is:

- **homogeneous** (single `dtype`)
- **N-dimensional**

- optimized for numeric computation

Key terms to know (names only; details come in later topics):

- `dtype` (data type)
- `shape` (dimensions)
- `ndim` (number of dimensions)
- `size` (total elements)

3. All Important Variations / Cases / Functions

Variation 1: Homogeneous dtype and automatic upcasting

NumPy arrays generally use one `dtype`. If you mix types, NumPy will convert to a common type when possible (e.g., ints + floats become floats).

Why it matters:

- predictable memory layout
- fast computation
- but you must be aware of precision/overflow (later topic)

Variation 2: Different behavior of operators on lists vs arrays

Some operators have very different meanings:

- on lists, `+` commonly means concatenation
- on arrays, `+` commonly means element-wise addition

Why it matters:

- prevents confusion when you switch from pure Python to NumPy

Variation 3: Views vs copies (motivation only)

NumPy often returns **views** when slicing (a different “window” into the same data), while Python list slicing creates a new list.

Why it matters:

- performance benefits
- can cause “unexpected” changes if you modify a view

Common functions you’ll encounter soon (names only)

- `np.array`
- `np.arange`
- `np.linspace`

2) Installing and Importing NumPy

1. Concept Explanation

Installation

NumPy is an external library, so it must be installed in your Python environment. The most common way is using `pip` (Python's package manager) or `conda` (if using Anaconda).

Importing

Once installed, you must bring it into your script using the `import` statement. By convention, the entire community imports it with the alias `np`.

Intuition

- **Installation:** Like downloading an app on your phone before you can use it.
- **Importing:** Like opening the app to start using its features.
- **Alias (`as np`):** Like a nickname. It's faster to type `np.array()` than `numpy.array()`.

2. Syntax / Definition

Terminal Commands (Installation)

```
# Using pip
pip install numpy

# Using conda
conda install numpy
```

Python Code (Importing)

```
import numpy as np
```

3. All Important Variations / Cases / Functions

Variation 1: Checking Version

To verify if NumPy is installed and check its version:

```
import numpy as np
print(np.__version__)
```

Variation 2: Import without Alias (Not Recommended)

```
import numpy
# Usage: numpy.array([1, 2, 3])
```

Variation 3: Importing Specific Functions (Use sparingly)

```
from numpy import array
# Usage: array([1, 2, 3])
```

4. Examples (VERY IMPORTANT)

Example 1: Basic Import and Verification

```
import numpy as np

# Check version (output will vary depending on your installation)
print("NumPy Version:", np.__version__)
```

Output

```
NumPy Version: 1.26.4
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

What is the command to install NumPy using pip?

Solution

```
pip install numpy
```

Q2 (Easy)

Write the standard way to import NumPy with its common alias.

Solution

```
import numpy as np
```

Q3 (Medium)

How can you check which version of NumPy is currently installed in your Python environment?

Solution

```
import numpy as np
print(np.__version__)
```

3) NumPy ndarray Basics

1. Concept Explanation

What is an ndarray?

The **ndarray** (N-dimensional array) is the heart of NumPy. It is a multidimensional container of items of the **same type and size**. Unlike Python lists, which can store mixed data types, an **ndarray** is strictly **homogeneous**.

Intuition: why "N-dimensional"?

- **1D (Vector)**: A single row of numbers (like a list).
- **2D (Matrix)**: Rows and columns (like a spreadsheet or a table).
- **3D (Tensor)**: A stack of 2D matrices (like a book where each page is a matrix).
- **ND**: NumPy can handle 4D, 5D, and beyond, which is crucial for complex ML models (e.g., video data is often 4D: time, height, width, channels).

2. Syntax / Definition

An **ndarray** is an instance of the class **numpy.ndarray**.

Basic Creation Syntax

```
import numpy as np
arr = np.array([data])
```

Key Internal Components

- **Data**: The actual raw bytes of the array.
- **Metadata**: Information about the data, such as its shape, number of dimensions (**ndim**), and data type (**dtype**).

3. All Important Variations / Cases / Functions

Case 1: Dimensionality (ndim)

- **0D Scalar**: A single number.
- **1D Array**: A sequence of numbers.

- **2D Array:** An array of 1D arrays.
- **3D Array:** An array of 2D arrays.

Case 2: Homogeneity (The strict rule)

If you provide mixed types (e.g., `[1, 2.5, "3"]`), NumPy will try to **coerce** them into a single type (usually the most flexible one, like string or float).

Case 3: Memory Layout

`ndarray` elements are stored in **contiguous memory**. This means they sit right next to each other, allowing the CPU to access them extremely fast compared to Python lists which store references scattered in memory.

Case 4: Data Types (dtype)

Every `ndarray` has a `dtype` (data type). Common dtypes include:

- `int32`, `int64`: Integers (32 or 64 bits)
- `float32`, `float64`: Floating point numbers
- `bool`: True/False
- `complex128`: Complex numbers
- `<U10`: Unicode string (max 10 characters)

4. Examples (VERY IMPORTANT)

Example 1: Creating different dimensions

```
import numpy as np

# 0D Array (Scalar)
arr0 = np.array(42)

# 1D Array (Vector)
arr1 = np.array([1, 2, 3])

# 2D Array (Matrix)
arr2 = np.array([[1, 2], [3, 4]])

# 3D Array (Tensor)
arr3 = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

print("0D:", arr0.ndim)
print("1D:", arr1.ndim)
print("2D:", arr2.ndim)
print("3D:", arr3.ndim)
```

Output

```
0D: 0
1D: 1
2D: 2
3D: 3
```

Example 2: Homogeneity (Coercion)

```
import numpy as np

# Mixed integers and floats -> All become floats
arr = np.array([1, 2.5, 3])
```



```
print(arr)
print(arr.dtype)
```

Output

```
[1.  2.5 3. ]
float64
```

Example 3: Nested structure check

```
import numpy as np

# A list of lists becomes a 2D array
data = [[10, 20], [30, 40]]
arr = np.array(data)

print(arr)
print("Type:", type(arr))
```

Output

```
[[10 20]
 [30 40]]
Type: <class 'numpy.ndarray'>
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

What does the `n` in `ndarray` stand for?

Solution

It stands for **N-dimensional**, indicating the array can have any number of dimensions.

Q2 (Easy)

If you create a NumPy array from the list `[1, "hello", 3.14]`, what will happen to the numbers?

Solution

NumPy will coerce all elements into a **string** type to maintain a homogeneous array (since strings are the most flexible type in this mix).

Q3 (Medium)

Create a 2D array with 2 rows and 3 columns using `np.array()`.

Solution

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr)
```

Q4 (Challenging)

How does the memory storage of an `ndarray` differ from a Python list, and why does this make NumPy faster?

Solution

- **Python lists** store pointers to objects scattered in memory. The CPU has to "jump" around to find each value.
- **NumPy arrays** store data in **contiguous blocks** of memory. The CPU can read the next value immediately, and it can perform "SIMD" (Single Instruction, Multiple Data) operations to process multiple numbers at once.

4) Creating Arrays (array, arange, linspace)

1. Concept Explanation

Introduction

NumPy provides several functions to create arrays from scratch or from existing Python sequences (like lists). Each method serves a specific purpose depending on whether you have the data already or need to generate a sequence of numbers.

Intuition

- `np.array()` : Use this when you already have the data in a list or tuple.
- `np.arange()` : Use this when you know the **step size** between numbers (like "give me every 2nd number from 0 to 10").
- `np.linspace()` : Use this when you know **how many numbers** you want in a range (like "give me exactly 5 numbers between 0 and 1").

2. Syntax / Definition

`np.array(object, dtype=None, ...)`

Creates an array from a Python list, tuple, or any array-like object.

`np.arange([start,] stop, [step,] dtype=None)`

Returns evenly spaced values within a given interval.

- `stop` is **exclusive** (just like Python's `range()`).

`np.linspace(start, stop, num=50, endpoint=True, ...)`

Returns `num` evenly spaced samples over the interval `[start, stop]` .

- `stop` is **inclusive** by default.

3. All Important Variations / Cases / Functions

Case 1: Specifying `dtype` in `np.array`

You can force a specific data type during creation:

```
arr = np.array([1, 2, 3], dtype='float32')
```

Case 2: `arange` with floating point steps

Unlike Python's `range()` , `np.arange()` supports float steps. However, due to floating-point precision, it's often safer to use `linspace` for float intervals.

Case 3: `linspace` endpoint

By setting `endpoint=False` , you can exclude the stop value.

4. Examples (VERY IMPORTANT)

Example 1: `np.array` from nested lists

```
import numpy as np
arr = np.array([[1, 2], [3, 4]])
print(arr)
```

Output

```
[[1 2]
```



```
[3 4]]
```

Example 2: `np.arange` with step

```
import numpy as np
# Start=0, Stop=10, Step=2
arr = np.arange(0, 10, 2)
print(arr)
```

Output

```
[0 2 4 6 8]
```

Example 3: `np.linspace` for precise counts

```
import numpy as np
# 5 numbers from 0 to 1 (inclusive)
arr = np.linspace(0, 1, 5)
print(arr)
```

Output

```
[0.    0.25 0.5   0.75 1.   ]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Create an array containing numbers from 10 to 50 with a step of 5.

Solution

```
np.arange(10, 51, 5)
```

Q2 (Medium)

Create an array of exactly 10 points between 0 and 5 (inclusive).

Solution

```
np.linspace(0, 5, 10)
```

Q3 (Challenging)

Why is `linspace` generally preferred over `arange` when working with floating-point numbers?

Solution

`arange` calculates the number of elements based on the step size, which can lead to precision errors with floats (potentially including or excluding the last element unexpectedly). `linspace` allows you to specify the exact number of elements, ensuring more predictable results for intervals.

5) Array Attributes (shape, ndim, dtype, size, itemsize)

1. Concept Explanation

Introduction

Attributes are properties of an `ndarray` that tell us about its structure and the data it holds. Unlike functions, attributes are accessed without parentheses (e.g., `arr.shape`).

Intuition

Think of an array as a package.

- `ndim`: How many layers of wrapping? (Dimensions)

- `shape` : What are the dimensions of the box? (Rows, Columns)
- `size` : How many items are inside total?
- `dtype` : What kind of items are they? (Integers, Floats)
- `itemsize` : How much space does one item take? (Bytes)

2. Syntax / Definition

`arr.ndim`

Returns the number of array dimensions.

`arr.shape`

Returns a tuple representing the dimensions (e.g., `(rows, cols)`).

`arr.size`

Returns the total number of elements in the array.

`arr.dtype`

Returns the data type of the elements.

`arr.itemsize`

Returns the size in bytes of each element.

3. All Important Variations / Cases / Functions

Case 1: Shape of 1D vs 2D

- 1D array shape: `(n,)`
- 2D array shape: `(n, m)`

Case 2: Total memory usage

The total memory consumed by an array can be calculated as `arr.size * arr.itemsize` .

4. Examples (VERY IMPORTANT)

Example 1: Basic Attribute Inspection

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])

print("Dimensions (ndim):", arr.ndim)
print("Shape:", arr.shape)
print("Total elements (size):", arr.size)
print("Data type (dtype):", arr.dtype)
print("Size of one element (itemsize):", arr.itemsize, "bytes")
```

Output

```
Dimensions (ndim): 2
Shape: (2, 3)
Total elements (size): 6
Data type (dtype): int64
Size of one element (itemsize): 8 bytes
```

Example 2: Checking itemsize for different dtypes

```
import numpy as np
a = np.array([1, 2], dtype='int32')
```

```
b = np.array([1, 2], dtype='int64')

print("int32 itemsize:", a.itemsize)
print("int64 itemsize:", b.itemsize)
```

Output

```
int32 itemsize: 4
int64 itemsize: 8
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

If an array `A` has a shape of `(3, 4, 5)`, what is its `ndim`?

Solution

`3` (because there are three values in the shape tuple).

Q2 (Medium)

Calculate the total number of elements in an array with shape `(5, 10, 2)` without using `.size`.

Solution

```
5 * 10 * 2 = 100
```

Q3 (Challenging)

You have an array of 1 million `float64` elements. How much memory (in MB) does it occupy? (1 MB = 1,000,000 bytes for simplicity).

Solution

```
float64 takes 8 bytes per element.
1,000,000 * 8 = 8,000,000 bytes = 8 MB .
```

PHASE 2: ARRAY CREATION METHODS

1) Zeros, Ones, and Full Arrays

1. Concept Explanation

Introduction

Sometimes you need to create an array of a specific shape but you don't have the data yet. NumPy provides functions to initialize arrays with specific values: 0s, 1s, or any constant value you choose.

Intuition

- `np.zeros()` : Think of this as creating a blank canvas or a placeholder for future calculations (like an accumulator).
- `np.ones()` : Useful for initializing weights in some algorithms or for creating a mask where everything is "active" by default.
- `np.full()` : When you need a specific starting value other than 0 or 1 (e.g., initializing a grid with -1 to indicate "empty").

2. Syntax / Definition

```
np.zeros(shape, dtype=float, ...)
```

Creates an array of the given shape filled with zeros. Default `dtype` is `float64`.

```
np.ones(shape, dtype=float, ...)
```

Creates an array of the given shape filled with ones. Default `dtype` is `float64`.

```
np.full(shape, fill_value, dtype=None, ...)
```

Creates an array of the given shape filled with `fill_value`.

3. All Important Variations / Cases / Functions

Case 1: Specifying Shape

The `shape` can be an integer (for 1D) or a tuple (for N-D).

- `np.zeros(5)` → 1D array of 5 zeros.
- `np.zeros((2, 3))` → 2D array of 2 rows and 3 columns.

Case 2: Data Types

By default, these functions create `float64` arrays. Always specify `dtype='int'` if you need integers to save memory.

Case 3: Empty and Uninitialized Arrays

- `np.empty(shape)`: Creates an array without initializing entries. It is the fastest method because it doesn't set any values, but the content is random (whatever was in memory).
- `np.full_like`, `np.zeros_like`, `np.ones_like`: Handled earlier, but very useful for matching existing array shapes.

4. Examples (VERY IMPORTANT)

Example 1: Creating Basic Arrays

```
import numpy as np

z = np.zeros((2, 2))
o = np.ones((3, 1), dtype=int)
f = np.full((2, 3), 7)

print("Zeros:\\n", z)
print("Ones:\\n", o)
print("Full:\\n", f)
```

Output

```
Zeros:
[[0.  0.]
 [0.  0.]]
Ones:
[[1]
 [1]
 [1]]
Full:
[[7 7 7]
 [7 7 7]]
```

Example 2: Using `_like` functions

```
import numpy as np

existing = np.array([[1, 2, 3], [4, 5, 6]])
new_zeros = np.zeros_like(existing)
```

```
print("New Zeros shape:", new_zeros.shape)
print(new_zeros)
```

Output

```
New Zeros shape: (2, 3)
[[0 0 0]
 [0 0 0]]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Create a 1D array of 10 zeros with an integer data type.

Solution

```
np.zeros(10, dtype=int)
```

Q2 (Medium)

Create a 3×3 matrix where every element is the number `9.9`.

Solution

```
np.full((3, 3), 9.9)
```

Q3 (Challenging)

You have an array `A` of shape `(100, 100)`. Create a new array `B` of the same shape filled with `1`s, but ensure it uses the same data type as `A`.

Solution

```
B = np.ones_like(A)
```

2) Identity Matrix and Eye Function

1. Concept Explanation

Introduction

In linear algebra, an **Identity Matrix** is a square matrix with 1s on the main diagonal and 0s elsewhere. It acts like the number "1" in matrix multiplication. NumPy provides two functions for this: `np.identity()` and `np.eye()`.

Intuition

- `np.identity()`: Strictly for **square** matrices (N x N).
- `np.eye()`: More flexible; can create non-square matrices and allows you to shift the diagonal of 1s.

2. Syntax / Definition

```
np.identity(n, dtype=None)
```

Returns a square identity array of size `n` x `n`.

```
np.eye(N, M=None, k=0, dtype=float)
```

Returns a 2D array with 1s on the k-th diagonal and 0s elsewhere.

- `N`: Number of rows.
- `M`: Number of columns (optional, defaults to `N`).
- `k`: Index of the diagonal (0 is the main diagonal, positive is upper, negative is lower).

3. All Important Variations / Cases / Functions

Case 1: Square vs Rectangular

`np.identity(3)` creates a 3×3 matrix.

`np.eye(3, 5)` creates a 3×5 matrix with 1s on the diagonal starting from (0,0).

Case 2: Shifting the Diagonal (`k` parameter)

By changing `k`, you can move the diagonal of 1s above or below the center.

4. Examples (VERY IMPORTANT)

Example 1: Identity vs Eye

```
import numpy as np

id_mat = np.identity(3)
eye_mat = np.eye(3, 4)

print("Identity 3x3:\\n", id_mat)
print("Eye 3x4:\\n", eye_mat)
```

Output

```
Identity 3x3:
[[1.  0.  0.]
 [0.  1.  0.]
 [0.  0.  1.]]
Eye 3x4:
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
```

Example 2: Shifting Diagonals

```
import numpy as np

upper = np.eye(4, k=1)
lower = np.eye(4, k=-1)

print("Upper Diagonal (k=1):\\n", upper)
print("Lower Diagonal (k=-1):\\n", lower)
```

Output

```
Upper Diagonal (k=1):
[[0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]
 [0.  0.  0.  0.]]
Lower Diagonal (k=-1):
[[0.  0.  0.  0.]
 [1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Create a 5×5 identity matrix.

Solution

`np.identity(5)` or `np.eye(5)`

Q2 (Medium)

Create a 4×4 matrix where the diagonal *above* the main diagonal is filled with 1s.

Solution

```
np.eye(4, k=1)
```

Q3 (Challenging)

What is the difference between `np.identity(4)` and `np.eye(4)` ?

Solution

`np.identity` can only create square matrices (N x N), whereas `np.eye` can create rectangular matrices (N x M) and allows shifting the diagonal using the `k` parameter.

3) Random Array Generation (rand, randn, randint, uniform)

1. Concept Explanation

Introduction

Random numbers are essential for initializing neural network weights, simulating data, or shuffling datasets. NumPy's `random` sub-module provides various distributions.

Intuition

- `rand`: Uniform distribution between 0 and 1.
- `randn`: Standard Normal distribution (Mean=0, StdDev=1).
- `randint`: Discrete random integers.
- `uniform`: Uniform distribution within a custom range `[low, high)`.

2. Syntax / Definition

```
np.random.rand(d0, d1, ..., dn)
```

Random values in a given shape from `[0, 1)`.

```
np.random.randn(d0, d1, ..., dn)
```

Random values from the "Standard Normal" distribution.

```
np.random.randint(low, high=None, size=None, dtype=int)
```

Random integers from `low` (inclusive) to `high` (exclusive).

```
np.random.uniform(low=0.0, high=1.0, size=None)
```

Random samples from a uniform distribution over `[low, high)`.

3. All Important Variations / Cases / Functions

Case 1: The New Generator API (Recommended for modern NumPy)

Modern NumPy (1.17+) recommends using `np.random.default_rng()` for better performance and statistical properties.

```
rng = np.random.default_rng()
arr = rng.random((3, 3))
```

Case 2: Setting a Seed

For reproducibility, you can set a seed:

```
np.random.seed(42)
```

4. Examples (VERY IMPORTANT)

Example 1: Comparing Distributions

```
import numpy as np

# Uniform [0, 1)
u = np.random.rand(2, 3)

# Normal (Gaussian)
n = np.random.randn(2, 3)

# Integers [0, 100)
i = np.random.randint(0, 100, size=(2, 3))

print("Uniform:\\n", u)
print("Normal:\\n", n)
print("Integers:\\n", i)
```

Output (Approximate)

```
Uniform:
[[0.44 0.96 0.01]
 [0.91 0.22 0.51]]
Normal:
[[-0.53  1.21 -0.12]
 [ 0.44 -0.82  0.11]]
Integers:
[[42 12 88]
 [ 7 55 23]]
```

Example 2: Custom Uniform Range

```
import numpy as np
# 5 random numbers between 10 and 20
arr = np.random.uniform(10, 20, size=5)
print(arr)
```

Output (Approximate)

```
[12.45 18.91 10.02 15.67 19.33]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Generate a single random integer between 1 and 10 (inclusive).

Solution

```
np.random.randint(1, 11)
```

Q2 (Medium)

Generate a 3×3 matrix of random numbers sampled from a Standard Normal distribution.

Solution

```
np.random.randn(3, 3)
```

Q3 (Challenging)

Why is `np.random.randn()` commonly used to initialize weights in Neural Networks?

Solution

Because it samples from a Gaussian distribution with a mean of 0. This helps in keeping the signal (activations) from exploding or vanishing early in training, as weights are centered around zero.

4) Reshaping Arrays (reshape)

1. Concept Explanation

Introduction

Reshaping means changing the arrangement of elements without changing the actual data. For example, turning a 1D array of 6 elements into a 2×3 matrix.

Intuition

Imagine you have 12 eggs. You can arrange them in one long row (1×12), two rows of six (2×6), or a carton of 3×4. The eggs (data) stay the same; only the box (shape) changes.

2. Syntax / Definition

`np.reshape(a, newshape)` or `arr.reshape(newshape)`

- `newshape`: An integer or a tuple of integers.
- Constraint**: The new shape must be compatible with the original number of elements (`size`).

3. All Important Variations / Cases / Functions

Case 1: The **1** Shortcut

You can use `-1` for **one** of the dimensions. NumPy will automatically calculate the correct value for that dimension based on the total size.

- `arr.reshape(2, -1)` → If size is 10, it becomes 2×5.

Case 2: Reshape vs Resize

- `reshape()` returns a **new view**; the original data is unchanged.
- `resize()` (the function `np.resize()`) returns a new array and can handle shape mismatches by repeating or truncating data. (The method `arr.resize()` modifies the array in-place).

4. Examples (VERY IMPORTANT)

Example 1: Basic Reshape

```
import numpy as np

arr = np.arange(1, 13) # 1 to 12
res = arr.reshape(3, 4)
```

```
print("Original:", arr.shape)
print("Reshaped:\\n", res)
```

Output

```
Original: (12,)
Reshaped:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

Example 2: Using the -1 shortcut

```
import numpy as np

arr = np.arange(12)
# We want 2 rows, let NumPy figure out the columns
res = arr.reshape(2, -1)

print("Shape:", res.shape)
```

Output

```
Shape: (2, 6)
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Can you reshape an array of 10 elements into a (3, 3) matrix? Why?

Solution

No. A (3, 3) matrix requires 9 elements. Reshaping must preserve the total number of elements.

Q2 (Medium)

Convert a 1D array of 20 elements into a 4×5 matrix using the `-1` shortcut.

Solution

```
arr.reshape(4, -1)
```

Q3 (Challenging)

What is the difference between `arr.reshape(size)` and `arr.flatten()` ?

Solution

`arr.reshape(size)` (where size is total elements) will turn any N-D array into a 1D array, but it returns a **view** (shared memory). `arr.flatten()` always returns a **copy** (separate memory).

5) Flatten vs Ravel

1. Concept Explanation

Introduction

Both functions are used to convert a multi-dimensional array into a 1D array. However, they handle memory differently.

Intuition

- `flatten()` : "Copy and Flatten". It creates a brand new 1D array. Changes to the flat array **do not** affect the original.
- `ravel()` : "Reveal the underlying data". It returns a flattened **view** whenever possible. Changes to the raveled array **do** affect the original.

2. Syntax / Definition

`arr.flatten(order='C')`

Returns a copy of the array collapsed into one dimension.

`arr.ravel(order='C')` or `np.ravel(a, order='C')`

Returns a contiguous flattened array.

3. All Important Variations / Cases / Functions

Case 1: Memory (Copy vs View)

- `flatten()` is slower because it allocates new memory.
- `ravel()` is faster because it usually points to the existing memory.

Case 2: The `order` parameter

- `'C'` : Row-major (C-style).
- `'F'` : Column-major (Fortran-style).

4. Examples (VERY IMPORTANT)

Example 1: Memory Behavior

```
import numpy as np

original = np.array([[1, 2], [3, 4]])

# Ravel (View)
rav = original.ravel()
rav[0] = 99

print("Modified via Ravel:\\n", original)

# Flatten (Copy)
flat = original.flatten()
flat[1] = 88

print("Modified via Flatten (No change to original):\\n", original)
```

Output

```
Modified via Ravel:
[[99  2]
 [ 3  4]]
Modified via Flatten (No change to original):
[[99  2]
 [ 3  4]]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Which function returns a copy: `ravel()` or `flatten()` ?

Solution

```
flatten()
```

Q2 (Medium)

When would you prefer `ravel()` over `flatten()` ?

Solution

When you are working with very large datasets and want to avoid the memory overhead of creating a copy, or when you actually want the changes made to the 1D array to reflect in the original N-D array.

Q3 (Challenging)

Given a 3×3 matrix, how do you flatten it in column-major order?

Solution

```
arr.flatten(order='F') or arr.ravel(order='F')
```

PHASE 3: INDEXING & SLICING

1) Basic Indexing (1D, 2D, 3D)

1. Concept Explanation

Introduction

Indexing is the process of accessing specific elements within an array. It uses a zero-based coordinate system.

Intuition

- **1D**: Like a single row of lockers. You just need one number (index) to open a locker.
- **2D**: Like a cinema hall. You need two numbers: a **Row** number and a **Seat** number.
- **3D**: Like a multi-story cinema complex. You need three numbers: **Floor**, **Hall (Row)**, and **Seat**.

2. Syntax / Definition

1D Indexing

```
arr[index]
```

2D Indexing

`arr[row_index, col_index]` or `arr[row_index][col_index]` *Note: `arr[r, c]` is preferred in NumPy for better performance.*

3D Indexing

```
arr[depth_index, row_index, col_index]
```

3. All Important Variations / Cases / Functions

Case 1: Negative Indexing

Just like Python lists, `-1` refers to the last element, `-2` to the second last, etc.

Case 2: Accessing Entire Rows/Columns

- `arr[1, :]` → Accesses all elements in the second row.
- `arr[:, 2]` → Accesses all elements in the third column.

Case 4: Ellipsis and NewAxis

- **Ellipsis (...)**: Used to select all remaining unspecified dimensions. For example, in a 5D array, `arr[0, ..., 0]` is equivalent to `arr[0, :, :, :, 0]`.
- **np.newaxis**: Used to increase the dimension of the existing array by one more dimension.
 - `arr[:, np.newaxis]` converts a 1D array of shape `(N,)` to a 2D array of shape `(N, 1)`.

4. Examples (VERY IMPORTANT)

Example 1: 1D and 2D Indexing

```
import numpy as np

# 1D
arr1 = np.array([10, 20, 30, 40])
print("1D second element:", arr1[1])

# 2D
arr2 = np.array([[1, 2, 3], [4, 5, 6]])
print("2D element at Row 1, Col 2:", arr2[1, 2])
```

Output

```
1D second element: 20
2D element at Row 1, Col 2: 6
```

Example 2: 3D Indexing

```
import numpy as np

arr3 = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])

# Accessing number 7: Second block (index 1), Second row (index 1), First column (index 0)
print("3D Element:", arr3[1, 1, 0])
```

Output

```
3D Element: 7
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

In a 1D array `arr = [5, 10, 15, 20]`, how do you access the last element using negative indexing?

Solution

```
arr[-1]
```

Q2 (Medium)

Given `A = np.array([[1, 2], [3, 4], [5, 6]])`, what is the code to access the number `4`?

Solution

```
A[1, 1]
```

Q3 (Challenging)

Create a 3D array of shape `(2, 2, 2)` and access the very first element of the second "page/block".

Solution

```
import numpy as np
arr = np.arange(8).reshape(2, 2, 2)
print(arr[1, 0, 0])
```

2) Slicing Arrays

1. Concept Explanation

Introduction

Slicing allows you to extract a sub-portion of an array. It follows the format `[start : stop : step]`.

Intuition

Think of a loaf of bread. Indexing is picking one crumb; slicing is cutting a whole chunk or taking every second slice for toast.

2. Syntax / Definition

Basic Slicing

```
arr[start : stop : step]
```

- `start`: Inclusive (default is 0)
- `stop`: Exclusive (default is length of array)
- `step`: Interval (default is 1)

3. All Important Variations / Cases / Functions

Case 1: Slicing 2D Arrays

```
arr[row_slice, col_slice]
```

- `arr[0:2, 1:3]` → Rows 0 to 1, Columns 1 to 2.

Case 2: Reversing an Array

```
arr[::-1]
```

 reverses the array.

Case 3: Omitting Indices

- `arr[:5]` → From start to index 4.
- `arr[2:]` → From index 2 to end.
- `arr[:]` → Entire array.

4. Examples (VERY IMPORTANT)

Example 1: 1D Slicing

```
import numpy as np

arr = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```



```
print("Slice 2 to 5:", arr[2:6])
print("Every second element:", arr[::2])
```

Output

```
Slice 2 to 5: [2 3 4 5]
Every second element: [0 2 4 6 8]
```

Example 2: 2D Slicing (Crucial for Data Science)

```
import numpy as np

matrix = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]
])

# Extract middle square: Rows 1-2, Columns 1-2
print("Sub-matrix:\\n", matrix[1:3, 1:3])
```

Output

```
Sub-matrix:
[[ 6  7]
 [10 11]]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Extract the first three elements of a 1D array `x`.

Solution

```
x[:3]
```

Q2 (Medium)

Given a 2D array, how do you extract only the last column?

Solution

```
arr[:, -1]
```

Q3 (Challenging)

How do you reverse the rows of a 2D matrix but keep the column order same?

Solution

```
matrix[::-1, :]
```

3) Boolean Indexing

1. Concept Explanation

Introduction

Boolean indexing allows you to select elements based on a **condition** rather than their position.

Intuition

Like a filter. "Show me all students with marks > 80". You don't care where they sit (index); you only care about their value.

2. Syntax / Definition

Basic Usage

`arr[condition]`

Example: `arr[arr > 5]` returns a 1D array of all elements greater than 5.

3. All Important Variations / Cases / Functions

Case 1: Multiple Conditions

Use `&` (and), `|` (or), and `~` (not). **Note: You must use parentheses around conditions.** `arr[(arr > 5) & (arr < 10)]`

Case 2: Filtering and Modifying

`arr[arr < 0] = 0` → Replaces all negative numbers with zero.

4. Examples (VERY IMPORTANT)

Example 1: Basic Filtering

```
import numpy as np

arr = np.array([1, 5, 8, 10, 12, 15])
mask = arr > 10
print("Mask:", mask)
print("Filtered elements:", arr[mask])
```

Output

```
Mask: [False False False False  True  True]
Filtered elements: [12 15]
```

Example 2: Complex Filtering

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
# Filter even numbers greater than 5
result = arr[(arr % 2 == 0) & (arr > 5)]
print(result)
```

Output

```
[ 6  8 10]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Given `arr = np.array([1, -2, 3, -4])`, write code to get only positive numbers.

Solution

```
arr[arr > 0]
```

Q2 (Medium)

Replace all elements in an array that are divisible by 3 with the number `-99`.

Solution

```
arr[arr % 3 == 0] = -99
```

Q3 (Challenging)

In a 2D array of scores, how would you select all scores that are between 40 and 60 (inclusive)?

Solution

```
scores[(scores >= 40) & (scores <= 60)]
```

4) Fancy Indexing

1. Concept Explanation

Introduction

Fancy Indexing means passing a **list of indices** to access multiple specific elements at once.

Intuition

Instead of slicing a range, you are "cherry-picking" specific items by their index numbers.

2. Syntax / Definition

1D Fancy Indexing

```
arr[[idx1, idx2, idx3]]
```

2D Fancy Indexing

```
arr[[row_indices], [col_indices]]
```

3. All Important Variations / Cases / Functions

Case 1: Order matters

The resulting array follows the order of indices provided in the list.

`arr[[3, 0, 1]]` will return elements in that exact order.

Case 2: 2D Fancy Indexing (Pairing)

`arr[[0, 1], [2, 3]]` picks elements at `(0, 2)` and `(1, 3)`. It does NOT pick a sub-matrix.

4. Examples (VERY IMPORTANT)

Example 1: 1D Picking

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60])
indices = [0, 4, 2]
print(arr[indices])
```

Output

```
[10 50 30]
```

Example 2: 2D Selection

```
import numpy as np

matrix = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

# Pick (0,0), (1,2), and (2,1)
print(matrix[[0, 1, 2], [0, 2, 1]])
```

Output

```
[1 6 8]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Given `arr = np.array([100, 200, 300, 400])`, use fancy indexing to get the 1st and 4th elements.

Solution

```
arr[[0, 3]]
```

Q2 (Medium)

How do you reorder an array `[1, 2, 3]` to `[3, 1, 2]` using fancy indexing?

Solution

```
arr[[2, 0, 1]]
```

Q3 (Challenging)

What is the difference between `matrix[0:2, 0:2]` and `matrix[[0, 1], [0, 1]]`?

Solution

`matrix[0:2, 0:2]` is **Slicing**: it returns a 2×2 sub-matrix.

`matrix[[0, 1], [0, 1]]` is **Fancy Indexing**: it returns a 1D array of elements at `(0,0)` and `(1,1)`.

5) Modifying Arrays using Indexing

1. Concept Explanation

Introduction

Indexing is not just for reading; it's for writing (assignment). You can change individual elements, slices, or filtered groups.

Intuition

Like finding a mistake in a document. You locate the line (index) and replace the word (value).

2. Syntax / Definition

Basic Assignment

```
arr[index] = new_value
```

3. All Important Variations / Cases / Functions

Case 1: Scalar Broadcast

Assigning one value to a whole slice: `arr[0:5] = 10`.

Case 2: Array-to-Slice Assignment

```
arr[0:3] = [10, 20, 30]
```

 . Sizes must match.

Case 3: Views vs Copies

Warning: Slicing returns a **view**. If you modify `sub_arr = arr[0:5]`, the original `arr` **also changes**. Fancy indexing returns a **copy**.

4. Examples (VERY IMPORTANT)

Example 1: Modifying Slices

```
import numpy as np

arr = np.zeros(5)
arr[1:4] = 9
print(arr)
```

Output

```
[0.  9.  9.  9.  0.]
```

Example 2: The "View" Trap

```
import numpy as np

original = np.array([1, 2, 3, 4, 5])
slice_view = original[0:3]
slice_view[0] = 99

print("Original after view modification:", original)
```

Output

```
Original after view modification: [99  2  3  4  5]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Set the first element of an array `x` to `0`.

Solution

```
x[0] = 0
```

Q2 (Medium)

Change all values in the second row of a 2D array to `100`.

Solution

```
arr[1, :] = 100
```

Q3 (Challenging)

How do you replace all negative values in an array with their absolute values?

Solution

```
arr[arr < 0] = np.abs(arr[arr < 0]) or simply arr = np.abs(arr) .
```

PHASE 4: OPERATIONS ON ARRAYS

1) Arithmetic Operations (element-wise)

1. Concept Explanation

Introduction

NumPy allows you to perform mathematical operations on arrays directly. These operations are performed **element-wise**, meaning the operation is applied to each corresponding pair of elements in the arrays.

Intuition

If you have two lists of numbers and you want to add them in pure Python, you'd need a loop. In NumPy, you just use `+`. It's like having two rows of soldiers and telling each pair to perform the same action simultaneously.

2. Syntax / Definition

Basic Arithmetic

- Addition: `a + b` or `np.add(a, b)`
- Subtraction: `a - b` or `np.subtract(a, b)`
- Multiplication: `a * b` or `np.multiply(a, b)`
- Division: `a / b` or `np.divide(a, b)`
- Exponentiation: `a ** b` or `np.power(a, b)`
- Modulo: `a % b` or `np.remainder(a, b)`

3. All Important Variations / Cases / Functions

Case 1: Array and Scalar

You can perform operations between an array and a single number (scalar). The scalar is applied to every element.

```
arr + 10
```

 adds 10 to every element.

Case 2: Array and Array

The arrays must have the same shape (or be compatible for broadcasting).

```
arr1 * arr2
```

 multiplies corresponding elements.

4. Examples (VERY IMPORTANT)

Example 1: Basic Operations

```
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

print("Sum:", a + b)
print("Product:", a * b)
print("Power:", a ** 2)
```

Output

```
Sum: [11 22 33 44]
Product: [ 10  40  90 160]
Power: [ 1  4  9 16]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Multiply every element in an array `x` by `5`.

Solution

```
x * 5
```

Q2 (Medium)

Subtract `arr1` from `arr2` element-wise.

Solution

```
arr2 - arr1
```

2) Broadcasting Concept (VERY IMPORTANT 🔥)

1. Concept Explanation

Introduction

Broadcasting is a powerful mechanism that allows NumPy to work with arrays of **different shapes** during arithmetic operations. Frequently, we have a smaller array and a larger array, and we want to use the smaller array multiple times to perform some operation on the larger array.

Intuition

Imagine you have a 3×3 grid of numbers (a matrix) and you want to add a single row `[1, 2, 3]` to every row of that grid. Instead of making three copies of the row, NumPy "broadcasts" the row across the matrix.

2. Syntax / Definition

Broadcasting follows a strict set of rules to determine if two arrays are compatible:

1. If the arrays do not have the same rank (number of dimensions), prepend the shape of the lower rank array with 1s until both shapes have the same length.
2. The two arrays are compatible in a dimension if they have the same size in that dimension, or if one of the arrays has size 1 in that dimension.
3. If this is not true, a `ValueError: operands could not be broadcast together` is raised.

Practical Visualization

- **Array A:** `(4, 3)`

- **Array B:** `(3,)` → prepended with 1 becomes `(1, 3)`
- **Compatibility:**
 - Dim 2: `3 == 3` (Match)
 - Dim 1: `4` vs `1` (1 matches anything)
- **Result:** Compatible! Shape will be `(4, 3)`.

3. All Important Variations / Cases / Functions

Case 1: Scalar Broadcasting

A scalar is always broadcastable to any shape.

```
np.zeros((3,3)) + 5
```

Case 2: 1D Array and 2D Array

A 1D array of shape `(3,)` can be broadcast over a 2D array of shape `(4, 3)`.

Case 3: Incompatible Shapes

Shape `(2, 3)` and `(2, 2)` cannot be broadcast together because `3 != 2` and neither is `1`.

4. Examples (VERY IMPORTANT)

Example 1: 1D to 2D Broadcasting

```
import numpy as np

matrix = np.array([[1, 1, 1], [2, 2, 2], [3, 3, 3]])
row = np.array([1, 0, 1])

result = matrix + row
print(result)
```

Output

```
[[2 1 2]
 [3 2 3]
 [4 3 4]]
```

Example 2: Column Broadcasting

```
import numpy as np

matrix = np.array([[1, 2, 3], [4, 5, 6]])
col = np.array([[10], [20]]) # Shape (2, 1)

print("Matrix:\\n", matrix)
print("Col:\\n", col)
print("Result:\\n", matrix + col)
```

Output

```
Matrix:
[[1 2 3]
 [4 5 6]]
Col:
[[10]
 [20]]
Result:
```



```
[[11 12 13]
 [24 25 26]]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Can you add a scalar `10` to a 3D array?

Solution

Yes. Scalars can be broadcast to any dimension.

Q2 (Medium)

Given `A` with shape `(4, 3)` and `B` with shape `(3,)`, will `A + B` work?

Solution

Yes. `B`'s shape will be treated as `(1, 3)`, which is compatible with `(4, 3)`.

Q3 (Challenging)

Given `A` with shape `(5, 1)` and `B` with shape `(1, 10)`, what will be the shape of `A + B`?

Solution

The resulting shape will be `(5, 10)`. Both arrays broadcast along their "1" dimension.

3) Comparison Operations

1. Concept Explanation

Introduction

Comparison operations compare elements of arrays and return a **Boolean array** (True/False).

Intuition

Like asking a question for every element: "Are you bigger than 10?". The answer for each element is either Yes (True) or No (False).

2. Syntax / Definition

- Greater than: `>`
- Less than: `<`
- Greater than or equal to: `>=`
- Less than or equal to: `<=`
- Equal to: `==`
- Not equal to: `!=`

3. All Important Variations / Cases / Functions

Case 1: Array and Scalar

Returns a boolean array of the same shape as the input array.

Case 2: Array and Array

Performs element-wise comparison. Arrays must be broadcastable.

4. Examples (VERY IMPORTANT)

Example 1: Basic Comparison

```
import numpy as np

arr = np.array([10, 20, 30, 40])
print(arr > 25)
print(arr == 20)
```

Output

```
[False False  True  True]
[False  True False False]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Create a boolean mask for elements in `arr` that are less than or equal to `5`.

Solution

```
arr <= 5
```

4) Logical Operations

1. Concept Explanation

Introduction

Logical operations are used to combine multiple boolean conditions. In NumPy, you use specific functions or bitwise operators rather than the standard Python `and`, `or`, and `not`.

Intuition

"Show me elements that are greater than 10 **AND** are even numbers."

2. Syntax / Definition

- Logical AND: `np.logical_and(cond1, cond2)` or `(cond1) & (cond2)`
- Logical OR: `np.logical_or(cond1, cond2)` or `(cond1) | (cond2)`
- Logical NOT: `np.logical_not(cond)` or `~(cond)`
- Logical XOR: `np.logical_xor(cond1, cond2)` or `(cond1) ^ (cond2)`

3. All Important Variations / Cases / Functions

Case 2: Bitwise vs Logical

For boolean NumPy arrays, `&`, `|`, and `~` behave as logical operations. Using the keyword `and` or `or` will throw an error on arrays because they expect a single truth value (a scalar boolean), not an array of booleans.

Case 3: The `where` Function (Conditional Selection)

`np.where(condition, x, y)`: Returns elements from `x` or `y` depending on `condition`.

- If condition is True, yield `x`, otherwise yield `y`.
- Extremely useful for replacing values based on a condition without modifying the original array in-place.

4. Examples (VERY IMPORTANT)

Example 1: Combining Conditions

```
import numpy as np

arr = np.array([1, 5, 10, 15, 20])
# Between 6 and 16
mask = (arr > 6) & (arr < 16)
print(arr[mask])
```

Output

```
[10 15]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Find elements that are either less than `2` or greater than `18` in `arr`.

Solution

```
arr[(arr < 2) | (arr > 18)]
```

5) Vectorization (Why it's powerful)

1. Concept Explanation

Introduction

Vectorization is the practice of replacing explicit Python loops with array expressions. In NumPy, operations are pushed down into highly optimized C and Fortran code.

Intuition

Imagine you have to sign 10,000 papers.

- **Looping:** You pick up each paper, sign it, put it down, and repeat 10,000 times. (Slow)
- **Vectorization:** You have a magic stamp that signs all 10,000 papers at once. (Fast)

2. Syntax / Definition

Any operation that avoids an explicit `for` loop in Python over array elements is considered vectorized.

3. All Important Variations / Cases / Functions

Why is it powerful?

1. **Speed:** Compiled code runs much faster than Python's interpreted loops.
2. **Readability:** Code looks more like mathematical equations.
3. **Optimizations:** NumPy can use SIMD (Single Instruction, Multiple Data) on modern CPUs.

4. Examples (VERY IMPORTANT)

Example 1: Loop vs Vectorization (Speed Concept)

```
import numpy as np
import time

size = 1000000
arr = np.arange(size)

# Looping (Pure Python style)
start = time.time()
result_loop = [x + 1 for x in arr]
print("Loop time:", time.time() - start)

# Vectorization (NumPy style)
start = time.time()
result_vec = arr + 1
print("Vectorized time:", time.time() - start)
```

Output (Approximate)

```
Loop time: 0.082 seconds
Vectorized time: 0.001 seconds
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Why does vectorization make NumPy faster?

Solution

It delegates the computation to highly optimized, pre-compiled C/Fortran routines, avoiding the overhead of Python's interpreter and dynamic typing during loops.

Q2 (Medium)

Rewrite this loop using vectorization:

```
# Loop version
res = []
for x in arr:
    res.append(x ** 2)
```

Solution

```
res = arr ** 2
```

 PHASE 5: MATHEMATICAL FUNCTIONS

1) Universal Functions (ufuncs)

1. Concept Explanation

Introduction

A **Universal Function** (or **ufunc**) is a function that operates on `ndarrays` in an element-by-element fashion. They are highly optimized, vectorized wrappers for simple mathematical operations.

Intuition

Think of a ufunc as a "broadcasting-aware" machine. When you feed it an array, it applies the function to every single cell simultaneously using pre-compiled C code, rather than waiting for a Python loop to visit each cell.

2. Syntax / Definition

Most ufuncs have a simple syntax:

```
np.function_name(array)
```

They can be:

- **Unary ufuncs:** Take one input (e.g., `np.sqrt(x)` , `np.exp(x)`).
- **Binary ufuncs:** Take two inputs (e.g., `np.add(x, y)` , `np.maximum(x, y)`).

3. All Important Variations / Cases / Functions

Case 1: Math Operations

Standard math like `np.sin` , `np.cos` , `np.exp` , `np.log` , `np.sqrt` .

Case 2: Comparison ufuncs

`np.greater` , `np.less` , `np.equal` (these power the `>` or `==` symbols).

Case 3: Performance

Ufuncs are much faster than using Python's `math` library inside a loop because they avoid "type checking" and "function call overhead" for every single element.

4. Examples (VERY IMPORTANT)

Example 1: Basic Unary ufunc

```
import numpy as np

arr = np.array([1, 4, 9, 16])
# Calculate square root of every element
result = np.sqrt(arr)
print(result)
```

Output

```
[1.  2.  3.  4.]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

What is the primary benefit of using a NumPy ufunc over a standard Python function in a loop?

Solution

Speed/Performance. ufuncs are implemented in compiled C code and use vectorization to process all elements at once.

2) Trigonometric Functions

1. Concept Explanation

Introduction

NumPy provides a complete set of trigonometric functions that work on arrays. **Important: All trigonometric functions in NumPy expect angles in RADIANS, not degrees.**

Intuition

Useful for signal processing, physics simulations, and any circular or wave-based data analysis.

2. Syntax / Definition

- Sine: `np.sin(arr)`
- Cosine: `np.cos(arr)`
- Tangent: `np.tan(arr)`
- Inverse: `np.arcsin()`, `np.arccos()`, `np.arctan()`

3. All Important Variations / Cases / Functions

Case 1: Degree to Radian Conversion

Since NumPy uses radians, you often need these:

- `np.radians(degrees)`
- `np.deg2rad(degrees)`
- `np.degrees(radians)`

4. Examples (VERY IMPORTANT)

Example 1: Sine of multiple angles

```
import numpy as np

angles_deg = np.array([0, 30, 45, 60, 90])
angles_rad = np.radians(angles_deg)

print("Sine values:", np.sin(angles_rad))
```

Output

```
Sine values: [0.          0.5         0.70710678 0.8660254  1.         ]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Calculate the cosine of 0, 90, and 180 degrees.

Solution

```
import numpy as np
deg = np.array([0, 90, 180])
print(np.cos(np.radians(deg)))
```

3) Logarithmic & Exponential Functions

1. Concept Explanation

Introduction

These functions are essential for scaling data (normalization), financial modeling, and complex scientific calculations.

2. Syntax / Definition

- Natural log (base e): `np.log(x)`
- Log base 10: `np.log10(x)`
- Log base 2: `np.log2(x)`
- Exponential (e^x): `np.exp(x)`

3. All Important Variations / Cases / Functions

Case 1: Handling Zero or Negatives

`np.log(0)` will return `-inf` and a warning. `np.log(-1)` will return `nan` (Not a Number).

4. Examples (VERY IMPORTANT)

Example 1: Exponential and Log

```
import numpy as np

arr = np.array([1, 2, 3])
print("e^x:", np.exp(arr))

data = np.array([1, 10, 100])
print("Log10:", np.log10(data))
```

Output

```
e^x: [ 2.71828183  7.3890561  20.08553692]
Log10: [0.  1.  2.]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Medium)

How do you calculate `log(x + 1)` for very small values of `x` to maintain precision?

Solution

Use `np.log1p(x)`. This is a special ufunc designed for accuracy when `x` is near zero.

4) Rounding Functions (ceil, floor, round)

1. Concept Explanation

Introduction

Rounding functions allow you to clean up floating point data by converting them to the nearest integer based on specific rules.

Intuition

- **Ceil:** Always "Rounds Up" to the next ceiling.
- **Floor:** Always "Rounds Down" to the floor.
- **Round:** Rounds to the nearest integer (even if it's lower).

2. Syntax / Definition

- `np.ceil(arr)`
- `np.floor(arr)`
- `np.round(arr, decimals=0)`
- `np.trunc(arr)` (Just removes the decimal part)

4. Examples (VERY IMPORTANT)

Example 1: Comparing Rounding Methods

```
import numpy as np

arr = np.array([-1.7, 1.5, 1.1, 2.8])

print("Floor:", np.floor(arr))
print("Ceil:", np.ceil(arr))
print("Round:", np.round(arr))
```

Output

```
Floor: [-2.  1.  1.  2.]
Ceil: [-1.  2.  2.  3.]
Round: [-2.  2.  1.  3.]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Round `3.14159` to 2 decimal places using NumPy.

Solution `np.round(3.14159, 2)`

5) Aggregation Functions (sum, mean, min, max, std)

1. Concept Explanation

Introduction

Aggregation functions take an array and return a single summary value (like the total or average).

Intuition

"Summarizing the data". Instead of looking at every row in a table, you just want to know the total sales or the average age.

2. Syntax / Definition

- `np.sum(arr)`
- `np.mean(arr)`
- `np.min(arr)` / `np.max(arr)`
- `np.std(arr)` (Standard Deviation)
- `np.var(arr)` (Variance)

3. All Important Variations / Cases / Functions

Case 2: Keepdims Parameter

Most aggregation functions have a `keepdims=True` argument. This keeps the dimensions of the original array, which is very useful for broadcasting after an aggregation.

Example: `np.mean(arr, axis=1, keepdims=True)` returns a column vector instead of a 1D array.

Case 3: Argmin and Argmax

- `np.argmin(arr)` : Returns the index of the minimum value.
- `np.argmax(arr)` : Returns the index of the maximum value.
Useful for finding "which student" had the highest marks, rather than "what" the mark was.

4. Examples (VERY IMPORTANT)

Example 1: Row and Column Sums

```
import numpy as np

matrix = np.array([[1, 2], [3, 4]])

print("Total Sum:", np.sum(matrix))
print("Sum along columns (axis 0):", np.sum(matrix, axis=0))
print("Sum along rows (axis 1):", np.sum(matrix, axis=1))
```

Output

```
Total Sum: 10
Sum along columns (axis 0): [4 6]
Sum along rows (axis 1): [3 7]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Medium)

In a 2D array of students' marks (rows=students, cols=subjects), how do you find the average mark of each student?

Solution `np.mean(marks_array, axis=1)`

Q2 (Challenging)

Find the difference between the maximum and minimum value in each column of a 2D array.

Solution `np.max(arr, axis=0) - np.min(arr, axis=0)`

PHASE 6: ARRAY MANIPULATION

1) Changing Shape (reshape, resize)

1. Concept Explanation

Introduction

Manipulation involves changing the structure of an existing array. While we touched on `reshape` earlier, it is important to understand how it differs from `resize`.

Intuition

- `reshape`: Like folding a piece of paper. The area (data) stays the same, but you change the dimensions. It returns a **view**.
- `resize`: Like cutting or adding more paper. It can change the total number of elements. It often modifies the array **in-place** or returns a new copy.

2. Syntax / Definition

`arr.reshape(new_shape)`

Changes shape without changing data. Must maintain the same `size`.

`arr.resize(new_shape)`

Changes shape and can change `size`.

- If the new size is larger, it fills with zeros (only if using `np.resize(arr, new_shape)`).
- If using `arr.resize(new_shape)`, it modifies the array in-place and requires the array to own its own data.

3. All Important Variations / Cases / Functions

Case 1: In-place vs New Array

- `np.reshape()` returns a new view.
- `arr.resize()` modifies in-place.
- `np.resize()` returns a new array with potentially repeated or zero-filled data.

4. Examples (VERY IMPORTANT)

Example 1: Reshape vs Resize

```
import numpy as np

arr = np.array([1, 2, 3, 4])

# Reshape
reshaped = arr.reshape(2, 2)
print("Reshaped (2x2):\n", reshaped)

# Resize (New array with np.resize)
resized = np.resize(arr, (3, 3))
print("Resized (3x3) - Data repeats:\n", resized)
```

Output

```
Reshaped (2x2):
[[1 2]
 [3 4]]
```



```
Resized (3x3) - Data repeats:  
[[1 2 3]  
 [4 1 2]  
 [3 4 1]]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

What happens if you try to `reshape` a 4-element array into a (3, 3) shape?

Solution

It will raise a `ValueError` because the total number of elements (4) does not match the required size (9).

2) Transpose and Flattening

1. Concept Explanation

Introduction

- **Transpose:** Swapping rows and columns.
- **Flattening:** Converting an N-dimensional array into a 1D array.

Intuition

- **Transpose:** Rotating a photo from portrait to landscape.
- **Flattening:** Taking all the items out of a multi-compartment box and laying them out in a single straight line.

2. Syntax / Definition

- Transpose: `arr.T` or `np.transpose(arr)`
- Flattening: `arr.flatten()` (returns a copy) or `arr.ravel()` (returns a view)

4. Examples (VERY IMPORTANT)

Example 1: Transpose

```
import numpy as np  
  
arr = np.array([[1, 2, 3], [4, 5, 6]])  
print("Original:\\n", arr)  
print("Transposed:\\n", arr.T)
```

Output

```
Original:  
[[1 2 3]  
 [4 5 6]]  
Transposed:  
[[1 4]  
 [2 5]  
 [3 6]]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

If an array `A` has shape `(2, 5)`, what is the shape of `A.T`?

Solution `(5, 2)`

3) Concatenation (concatenate, vstack, hstack)

1. Concept Explanation

Introduction

Joining two or more arrays together.

Intuition

- `concatenate`: General purpose joining along a specified axis.
- `vstack` (**Vertical Stack**): Stacking arrays on top of each other (like building a tower).
- `hstack` (**Horizontal Stack**): Stacking arrays side-by-side (like a train).

2. Syntax / Definition

- `np.concatenate((a, b), axis=0)`
- `np.vstack((a, b))`
- `np.hstack((a, b))`
- `np.dstack((a, b))` (Depth Stack)

Case 2: `tile` and `repeat`

- `np.tile(arr, reps)`: Repeats the entire array.
- `np.repeat(arr, repeats, axis=None)`: Repeats elements of an array.

4. Examples (VERY IMPORTANT)

Example 1: Stacking

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print("Vertical Stack:\\n", np.vstack((a, b)))
print("Horizontal Stack:", np.hstack((a, b)))
```

Output

```
Vertical Stack:
[[1 2 3]
 [4 5 6]]
Horizontal Stack: [1 2 3 4 5 6]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Medium)

Combine two 2D arrays side-by-side.

Solution `np.hstack((arr1, arr2))` or `np.concatenate((arr1, arr2), axis=1)`

4) Splitting Arrays

1. Concept Explanation

Introduction

Splitting is the opposite of concatenation. It breaks one array into multiple smaller arrays.

2. Syntax / Definition

- `np.split(arr, sections)`
- `np.vsplit(arr, sections)` (Split vertically)
- `np.hsplit(arr, sections)` (Split horizontally)

4. Examples (VERY IMPORTANT)

Example 1: Horizontal Split

```
import numpy as np

arr = np.arange(16).reshape(4, 4)
# Split into 2 equal parts horizontally
parts = np.hsplit(arr, 2)

print("First Part:\n", parts[0])
print("Second Part:\n", parts[1])
```

Output

```
First Part:
[[ 0  1]
 [ 4  5]
 [ 8  9]
[12 13]]
Second Part:
[[ 2  3]
 [ 6  7]
[10 11]
[14 15]]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Medium)

Split an array of 10 elements into 5 equal parts.

Solution `np.split(arr, 5)`

5) Adding and Removing Elements (append, insert, delete)

1. Concept Explanation

Introduction

These functions allow you to grow or shrink an array. Note: Unlike Python lists, these operations are **not in-place** and return a new array.

2. Syntax / Definition

- `np.append(arr, values, axis=None)`
- `np.insert(arr, obj, values, axis=None)`
- `np.delete(arr, obj, axis=None)`

4. Examples (VERY IMPORTANT)

Example 1: Append and Delete

```
import numpy as np

arr = np.array([1, 2, 3])

# Append
arr = np.append(arr, [4, 5])
print("After append:", arr)

# Delete element at index 1
arr = np.delete(arr, 1)
print("After delete index 1:", arr)
```

Output

```
After append: [1 2 3 4 5]
After delete index 1: [1 3 4 5]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Insert the value `99` at index `2` of array `x`.

Solution `np.insert(x, 2, 99)`

PHASE 7: LINEAR ALGEBRA (VERY IMPORTANT)

1) Introduction to Linear Algebra in NumPy

1. Concept Explanation

Introduction

Linear Algebra is the mathematical foundation of Data Science, Machine Learning, and Graphics. In NumPy, 2D arrays are treated as **matrices**, and 1D arrays as **vectors**.

Intuition

Think of Linear Algebra as a way to solve many equations at once. Instead of calculating one number at a time, we manipulate entire sets of data (matrices) to find patterns, rotate images, or train neural networks.

2. Syntax / Definition

NumPy provides the `numpy.linalg` module for advanced linear algebra operations, but basic operations like dot products are available in the main `numpy` namespace.

3. All Important Variations / Cases / Functions

Case 1: The `linalg` Module

Most complex operations (inverse, eigenvalues, etc.) reside in `np.linalg`.

Case 2: Matrix vs Array

While NumPy has a `np.matrix` class, it is **deprecated**. Always use `np.ndarray` (standard arrays) for matrix operations.

Case 3: Solving Systems of Linear Equations

- `np.linalg.solve(A, b)`: Solves a system of linear equations $Ax = b$ for x . This is more efficient and numerically stable than computing `x = inv(A) @ b`.

Case 4: Norms

- `np.linalg.norm(x, ord=None)`: Computes the norm of a vector or matrix.
 - `ord=2`: L2 norm (Euclidean distance, default)
 - `ord=1`: L1 norm (Manhattan distance)
 - `ord=np.inf`: Infinity norm
 - `ord='fro'`: Frobenius norm (for matrices)

Case 5: Matrix Rank

- `np.linalg.matrix_rank(A)`: Computes the rank of a matrix (number of linearly independent rows/columns).

4. Examples (VERY IMPORTANT)

Example 1: Basic Matrix Representation

```
import numpy as np

# A 2x2 Matrix
A = np.array([[1, 2], [3, 4]])
# A Vector
v = np.array([10, 20])

print("Matrix A:\\n", A)
print("Vector v:", v)
```

Output

```
Matrix A:
[[1 2]
 [3 4]]
Vector v: [10 20]
```

Example 2: Solving a System of Equations

```
import numpy as np

# Solve: 2x + 3y = 1, 4x + 1y = -5
A = np.array([[2, 3], [4, 1]])
b = np.array([1, -5])
x = np.linalg.solve(A, b)
print("Solution x, y:", x)
```

Output

```
Solution x, y: [-2.  1.]
```

Example 3: Vector Norms

```
import numpy as np

v = np.array([3, 4])
l2_norm = np.linalg.norm(v) # Default
l1_norm = np.linalg.norm(v, ord=1)
linf_norm = np.linalg.norm(v, ord=np.inf)

print(f"L2 Norm: {l2_norm}")
print(f"L1 Norm: {l1_norm}")
print(f"L-inf Norm: {linf_norm}")
```

Output

```
L2 Norm: 5.0
L1 Norm: 7.0
L-inf Norm: 4.0
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

What module in NumPy is used for linear algebra?

Solution

```
numpy.linalg
```

2) Dot Product & Matrix Multiplication

1. Concept Explanation

Introduction

The **Dot Product** is a fundamental operation. For 1D arrays, it's the sum of products of corresponding elements. For 2D arrays, it's **Matrix Multiplication**.

Intuition

- **1D Dot Product:** A "similarity" score between two vectors.

- **2D Matrix Multiplication:** A way to transform data from one space to another (e.g., rotating or scaling a coordinate).

2. Syntax / Definition

- Dot Product (1D/2D): `np.dot(a, b)`
- Matrix Multiplication (Preferred): `a @ b` or `np.matmul(a, b)` *Note: `@` is the modern Python operator for matrix multiplication.*

3. All Important Variations / Cases / Functions

Case 1: Dimensionality Rule

For `A @ B` to work, the number of **columns in A** must equal the number of **rows in B**.
Resulting shape = (Rows of A, Columns of B).

4. Examples (VERY IMPORTANT)

Example 1: 1D Dot Product

```
import numpy as np

v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])

# (1*4) + (2*5) + (3*6) = 4 + 10 + 18 = 32
print("Dot Product:", np.dot(v1, v2))
```

Output

```
Dot Product: 32
```

Example 2: 2D Matrix Multiplication

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Using @ operator
result = A @ B
print("A @ B:\\n", result)
```

Output

```
A @ B:
[[19 22]
 [43 50]]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

If `A` has shape `(2, 3)` and `B` has shape `(3, 4)`, what is the shape of `A @ B`?

Solution

`(2, 4)`

Q2 (Medium)

What happens if you use `*` instead of `@` on two 2D arrays?

Solution

`*` performs **element-wise multiplication**, not matrix multiplication. Matrix multiplication requires `@` or `np.dot()`.

3) Determinant

1. Concept Explanation

Introduction

The **Determinant** is a scalar value that can be computed from a square matrix. It tells us if a matrix can be inverted and how it scales area/volume.

Intuition

If the determinant is **0**, the matrix "squashes" space into a lower dimension (like a line or a point), and it **cannot be inverted**.

2. Syntax / Definition

- `np.linalg.det(matrix)`

4. Examples (VERY IMPORTANT)

Example 1: Calculating Determinant

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
# (1*4) - (2*3) = 4 - 6 = -2
det = np.linalg.det(A)
print("Determinant:", round(det, 2))
```

Output

```
Determinant: -2.0
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

If `det(A) = 0`, what does that tell you about the inverse of `A`?

Solution

The inverse **does not exist** (the matrix is singular).

4) Inverse of Matrix

1. Concept Explanation

Introduction

The **Inverse** of a matrix `A` is a matrix `A-1` such that `A @ A-1 = I` (Identity Matrix).

Intuition

Inverting a matrix is like "undoing" a transformation. If a matrix moves a point, the inverse moves it back to where it started.

2. Syntax / Definition

- `np.linalg.inv(matrix)`

4. Examples (VERY IMPORTANT)

Example 1: Inverting and Verifying

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
A_inv = np.linalg.inv(A)

print("Inverse:\\n", A_inv)

# Verify: A @ A_inv should be Identity
print("Verification:\\n", np.round(A @ A_inv))
```

Output

```
Inverse:
[[-2.   1. ]
 [ 1.5 -0.5]]
Verification:
[[1.  0.]
 [0.  1.]]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Medium)

What error does NumPy throw if you try to invert a singular matrix (det=0)?

Solution

```
LinAlgError: Singular matrix
```

5) Eigenvalues and Eigenvectors

1. Concept Explanation

Introduction

For a matrix `A`, an **Eigenvector** is a vector that doesn't change direction when the transformation `A` is applied. It only gets scaled by a factor called the **Eigenvalue**.

Intuition

Think of a spinning globe. The axis of rotation is an eigenvector because points on the axis don't move "off-axis"; they stay in place (scale=1). In Data Science, this is the basis of **PCA (Principal Component Analysis)**.

2. Syntax / Definition

- `eigenvalues, eigenvectors = np.linalg.eig(matrix)`

4. Examples (VERY IMPORTANT)

Example 1: Finding Eigen-pairs

```
import numpy as np

A = np.array([[1, 2], [2, 1]])
vals, vecs = np.linalg.eig(A)

print("Eigenvalues:", vals)
print("Eigenvectors:\\n", vecs)
```

Output

```
Eigenvalues: [ 3. -1.]
Eigenvectors:
[[ 0.70710678 -0.70710678]
 [ 0.70710678  0.70710678]]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Challenging)

Why are eigenvalues important in Machine Learning?

Solution

They help in **Dimensionality Reduction**. In techniques like PCA, eigenvalues tell us which directions (eigenvectors) contain the most information (variance). We keep the eigenvectors with the largest eigenvalues.

PHASE 8: STATISTICS

1) Mean, Median, Standard Deviation

1. Concept Explanation

Introduction

Statistical functions allow us to summarize large datasets into a few meaningful numbers. This is the first step in any Data Science or ML project (Exploratory Data Analysis).

Intuition

- **Mean:** The "Average". If you distributed all values equally, what would each person get?
- **Median:** The "Middle". If everyone stood in a line from shortest to tallest, who is exactly in the center? (Crucial when you have outliers like a billionaire in a room of middle-class people).
- **Standard Deviation (Std):** The "Spread". How far, on average, are the numbers from the mean? A low Std means data is clustered; high Std means it's scattered.

2. Syntax / Definition

- `np.mean(arr)`
- `np.median(arr)`
- `np.std(arr)`

3. All Important Variations / Cases / Functions

Case 1: Mean vs Median with Outliers

- Mean is sensitive to outliers.
- Median is robust (resistant) to outliers.

Case 2: Standard Deviation vs Variance

Std is just the square root of Variance. It is preferred because it's in the same units as the original data.

Case 3: Quantiles

- `np.quantile(arr, q)` : Similar to `percentile` , but `q` is a value between 0 and 1. For example, `np.quantile(arr, 0.5)` is the median.

Case 4: Peak-to-Peak (ptp)

- `np.ptp(arr)` : The range of the data (max - min). Useful for understanding the spread.

4. Examples (VERY IMPORTANT)

Example 1: Calculating Statistics

```
import numpy as np

data = np.array([10, 20, 30, 40, 500]) # 500 is an outlier

print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Std Deviation:", np.round(np.std(data), 2))
print("Peak-to-Peak (Range):", np.ptp(data))
print("Quantile (0.25):", np.quantile(data, 0.25))
```

Output

```
Mean: 120.0
Median: 30.0
Std Deviation: 190.26
Peak-to-Peak (Range): 490
Quantile (0.25): 20.0
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Calculate the mean of an array `[1, 2, 3, 4, 5]` .

Solution `np.mean([1, 2, 3, 4, 5])` → `3.0`

2) Variance and Percentiles

1. Concept Explanation

Introduction

- **Variance:** Measures how far each number in the set is from the mean and thus from every other number in the set.
- **Percentiles:** Indicates the value below which a given percentage of observations in a group of observations falls.

Intuition

- **Variance:** If you're shooting at a target, variance is the average squared distance of your hits from the bullseye.
- **Percentiles:** If you are in the 90th percentile of a test, you scored better than 90% of the people.

2. Syntax / Definition

- Variance: `np.var(arr)`
- Percentile: `np.percentile(arr, q)` where `q` is the percentile (0-100).

4. Examples (VERY IMPORTANT)

Example 1: Percentiles in Action

```
import numpy as np

scores = np.array([40, 50, 60, 70, 80, 90, 100])

print("50th Percentile (Median):", np.percentile(scores, 50))
print("90th Percentile:", np.percentile(scores, 90))
```

Output

```
50th Percentile (Median): 70.0
90th Percentile: 94.0
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Medium)

Find the 25th (Q1) and 75th (Q3) percentiles of an array.

Solution

```
np.percentile(arr, [25, 75])
```

3) Correlation Coefficient

1. Concept Explanation

Introduction

The correlation coefficient measures the **strength and direction** of a linear relationship between two variables. It ranges from `-1` to `+1`.

Intuition

- `+1`: Perfect positive relationship (As X goes up, Y goes up).
- `-1`: Perfect negative relationship (As X goes up, Y goes down).
- `0`: No linear relationship.

2. Syntax / Definition

- `np.corrcoef(x, y)`
Returns a correlation matrix. The values at `[0, 1]` and `[1, 0]` are the correlation between `x` and `y`.

4. Examples (VERY IMPORTANT)

Example 1: Correlation between Height and Weight

```
import numpy as np

height = np.array([150, 160, 170, 180])
weight = np.array([50, 62, 75, 85])

corr_matrix = np.corrcoef(height, weight)
print("Correlation Matrix:\\n", corr_matrix)
print("Correlation coefficient:", round(corr_matrix[0, 1], 4))
```

Output

```
Correlation Matrix:
[[1.          0.99596041]
 [0.99596041  1.          ]]
Correlation coefficient: 0.996
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Challenging)

What does a correlation of `0.05` suggest between two variables?

Solution

It suggests a **very weak** or almost non-existent linear relationship.

4) Axis-based Operations

1. Concept Explanation

Introduction

Statistical operations can be applied to the whole array, or specifically along rows or columns in a 2D/3D array.

Intuition

In a table of student marks (Rows = Students, Cols = Subjects):

- `axis=0`: Average marks **per subject** (vertical).
- `axis=1`: Average marks **per student** (horizontal).

2. Syntax / Definition

Most stats functions accept an `axis` parameter.

- `axis=0`: Down the columns.
- `axis=1`: Across the rows.

4. Examples (VERY IMPORTANT)

Example 1: Row/Column Mean

```
import numpy as np

marks = np.array([
    [80, 90, 70],
    [60, 70, 80]
])

print("Mean per subject (axis 0):", np.mean(marks, axis=0))
print("Mean per student (axis 1):", np.mean(marks, axis=1))
```

Output

```
Mean per subject (axis 0): [70. 80. 75.]
Mean per student (axis 1): [80. 70.]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Medium)

Find the maximum value in each row of a 2D array.

Solution

```
np.max(arr, axis=1)
```

PHASE 9: ADVANCED NUMPY

1) Copy vs View (VERY IMPORTANT ⚠)

1. Concept Explanation

Introduction

When you manipulate an array (like slicing), NumPy tries to be efficient by not making a new copy of the data. Instead, it creates a **View**. Understanding the difference between a Copy and a View is critical to avoid accidental data modification.

Intuition

- **View:** A "window" or "shortcut" to the original data. If you change the data through the window, the original data is changed.
- **Copy:** A completely new, independent array with its own memory. Changes here do not affect the original.

2. Syntax / Definition

- **View:** Created by slicing (`arr[0:5]`) or `.view()` .
- **Copy:** Created by `.copy()` .

3. All Important Variations / Cases / Functions

Case 1: Checking for View

You can check if an array owns its data using `arr.base`.

- If `arr.base` is `None`, it's a **Copy**.
- If `arr.base` is the original array, it's a **View**.

4. Examples (VERY IMPORTANT)

Example 1: View Modification

```
import numpy as np

original = np.array([1, 2, 3, 4, 5])
view_arr = original[0:3]

view_arr[0] = 99
print("Original after view edit:", original)
```

Output

```
Original after view edit: [99  2  3  4  5]
```

Example 2: Copy Independence

```
import numpy as np

original = np.array([1, 2, 3, 4, 5])
copy_arr = original.copy()

copy_arr[0] = 99
print("Original after copy edit:", original)
print("Copy array:", copy_arr)
```

Output

```
Original after copy edit: [1 2 3 4 5]
Copy array: [99  2  3  4  5]
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Does slicing `arr[1:4]` create a copy or a view?

Solution

It creates a **View**.

2) Memory Layout (C vs F order)

1. Concept Explanation

Introduction

NumPy arrays are stored in contiguous memory blocks. The "Order" defines how multi-dimensional arrays are flattened into that linear memory.

Intuition

- **C-order (Row-major)**: Elements are stored row by row (Standard in Python/C).
- **F-order (Column-major)**: Elements are stored column by column (Standard in Fortran/MATLAB).

2. Syntax / Definition

- `order='C'` : C-style (Row-major)
- `order='F'` : Fortran-style (Column-major)

3. All Important Variations / Cases / Functions

Why does it matter?

Performance. Accessing elements in the same order they are stored in memory is much faster due to CPU caching.

Case 2: Checking and Changing Order

- `arr.flags['C_CONTIGUOUS']` : Checks if array is C-contiguous.
- `arr.flags['F_CONTIGUOUS']` : Checks if array is F-contiguous.
- `np.ascontiguousarray(arr)` : Returns a C-contiguous copy if not already.
- `np.asfortranarray(arr)` : Returns an F-contiguous copy if not already.

4. Examples (VERY IMPORTANT)

Example 1: Creating arrays with specific order

```
import numpy as np

# C-order (Default)
c_arr = np.array([[1, 2], [3, 4]], order='C')
# F-order
f_arr = np.array([[1, 2], [3, 4]], order='F')

print("C-order flags:\n", c_arr.flags['C_CONTIGUOUS'])
print("F-order flags:\n", f_arr.flags['F_CONTIGUOUS'])
```

Output

```
C-order flags: True
F-order flags: True
```

Example 2: Forcing Contiguity

```
import numpy as np

# A non-contiguous slice
arr = np.array([[1, 2, 3], [4, 5, 6]])
slice_arr = arr[:, 1] # This is not C or F contiguous
```



```
print("Is slice C-contiguous?", slice_arr.flags['C_CONTIGUOUS'])
print("Is slice F-contiguous?", slice_arr.flags['F_CONTIGUOUS'])

# Make it contiguous
c_contig = np.ascontiguousarray(slice_arr)
print("Is new array C-contiguous?", c_contig.flags['C_CONTIGUOUS'])
```

Output

```
Is slice C-contiguous? False
Is slice F-contiguous? False
Is new array C-contiguous? True
```

3) Iterating Arrays (nditer)

1. Concept Explanation

Introduction

While you can use standard Python loops, `np.nditer` is a sophisticated multidimensional iterator object to efficiently visit every element of an array.

Intuition

It's a "smart" loop that handles complex memory layouts and broadcasting automatically.

2. Syntax / Definition

```
for x in np.nditer(arr):
    print(x)
```

4. Examples (VERY IMPORTANT)

Example 1: Basic Iteration

```
import numpy as np

arr = np.array([[1, 2], [3, 4]])
for x in np.nditer(arr):
    print(x, end=' ')
```

Output

```
1 2 3 4
```

4) Structured Arrays

1. Concept Explanation

Introduction

Structured arrays are ndarrays whose datatype is a composition of simpler datatypes organized as a sequence of named fields.

Intuition

It's like a mini-database or a basic version of a Pandas DataFrame. Each element (row) has named columns with different types (e.g., Name=String, Age=Int).

2. Syntax / Definition

```
dtype = [('name', 'S10'), ('age', 'i4'), ('weight', 'f4')]
arr = np.array([('Alice', 25, 55.5), ('Bob', 30, 80.2)], dtype=dtype)
```

4. Examples (VERY IMPORTANT)

Example 1: Accessing Fields

```
import numpy as np

data = np.array([('Alice', 25), ('Bob', 30)],
                dtype=[('name', 'U10'), ('age', 'i4')])

print("Names:", data['name'])
print("First person's age:", data[0]['age'])
```

Output

```
Names: ['Alice' 'Bob']
First person's age: 25
```

Example 2: Complex Structured Array

```
import numpy as np

# Define a more complex dtype
dt = np.dtype([('name', 'U10'),
               ('grades', 'f4', (3,)), # 3 grades per person
               ('average', 'f4')])

students = np.array([('Alice', [8.5, 9.0, 7.5], 8.33),
                     ('Bob', [6.0, 7.5, 8.0], 7.17)], dtype=dt)

print("All grades for Alice:", students[0]['grades'])
print("Average for all students:", students['average'])
```

Output

```
All grades for Alice: [8.5 9.  7.5]
Average for all students: [8.33 7.17]
```

5) Masked Arrays

1. Concept Explanation

Introduction

Masked arrays are arrays that may have missing or invalid entries. `numpy.ma` module provides a way to handle this by "masking" those values so they are ignored in computations.

Intuition

Like putting a piece of tape over a broken sensor's reading so it doesn't mess up your average calculation.

2. Syntax / Definition

```
import numpy.ma as ma
mx = ma.masked_array(data, mask=[False, True, False])
```

4. Examples (VERY IMPORTANT)

Example 1: Masking invalid data

```
import numpy as np
import numpy.ma as ma

x = np.array([1, 2, -999, 4])
# Mask the value -999
mx = ma.masked_array(x, mask=[0, 0, 1, 0])

print("Mean with mask:", mx.mean())
```

Output

```
Mean with mask: 2.3333333333333335
```

PHASE 10: REAL-WORLD USAGE

1) Working with CSV using NumPy

1. Concept Explanation

Introduction

CSV (Comma Separated Values) is the most common format for tabular data. NumPy provides built-in functions to load data from these files directly into `ndarrays`.

Intuition

Think of it as importing an Excel sheet into a format that Python can perform lightning-fast math on. While Pandas is often used for this, NumPy's `genfromtxt` and `loadtxt` are powerful for large, purely numeric datasets.

2. Syntax / Definition

```
np.loadtxt(fname, delimiter, skiprows, ...)
```

Best for simple, clean files where all data is the same type.

```
np.genfromtxt(fname, delimiter, skip_header, filling_values, ...)
```

More robust; handles missing values and complex data structures.

3. All Important Variations / Cases / Functions

Case 1: Saving Data

Use `np.savetxt('file.csv', arr, delimiter=',')` to export your results back to a CSV.

Case 2: Specifying Data Types

You can use `dtype=None` with `genfromtxt` to let NumPy guess the types.

4. Examples (VERY IMPORTANT)

Example 1: Loading a simple CSV

```
import numpy as np

# Suppose 'data.csv' contains:
# 1, 2, 3
# 4, 5, 6

# data = np.loadtxt('data.csv', delimiter=',')
# print(data)
```

5. Practice Questions (from easy to challenging) with solution

Q1 (Easy)

Which NumPy function is better for handling CSV files with missing values?

Solution

`np.genfromtxt` is more robust for missing data compared to `np.loadtxt`.

2) Handling Missing Values

1. Concept Explanation

Introduction

Real-world data is messy. It often contains `NaN` (Not a Number) or `inf` (Infinity). NumPy has specialized functions to find and handle these values.

2. Syntax / Definition

- `np.isnan(arr)` : Returns a boolean mask where `True` indicates NaN.
- `np.isinf(arr)` : Returns a boolean mask for infinite values.
- `np.nan_to_num(arr)` : Replaces NaNs with zero and infs with large numbers.

3. All Important Variations / Cases / Functions

Case 1: Ignoring NaNs in Stats

Standard functions like `np.mean()` return `nan` if the array has even one `nan`. Use "nan-safe" versions:

- `np.nanmean()`
- `np.nansum()`

4. Examples (VERY IMPORTANT)

Example 1: Calculating Mean with NaNs

```
import numpy as np

data = np.array([1, 2, np.nan, 4])

print("Standard mean:", np.mean(data))
print("Safe mean:", np.nanmean(data))
```

Output

```
Standard mean: nan
Safe mean: 2.3333333333333335
```

3) Performance Comparison (NumPy vs Python list)

1. Concept Explanation

Introduction

NumPy is faster because of Vectorization and fixed-type memory.

Intuition

- **Python List:** A list of "anywhere" objects. Adding two lists requires checking the type of every single element and finding its location in memory one by one.
- **NumPy Array:** A solid block of numbers. Adding two arrays is a single instruction sent to the CPU to process the entire block.

4. Examples (VERY IMPORTANT)

Example 1: Timing a large operation

```
import numpy as np
import time

# 1 million elements
size = 1000000
L1 = list(range(size))
L2 = list(range(size))
A1 = np.arange(size)
A2 = np.arange(size)

# Python List
start = time.time()
res_l = [x + y for x, y in zip(L1, L2)]
print("List time:", time.time() - start)

# NumPy
start = time.time()
res_a = A1 + A2
print("NumPy time:", time.time() - start)
```

Output (Approximate)

```
List time: 0.095 seconds
NumPy time: 0.002 seconds
```

4) NumPy with Pandas (connection overview)

1. Concept Explanation

Introduction

Pandas is built **on top** of NumPy. A Pandas Series is essentially a NumPy array with an index (labels), and a DataFrame is a collection of NumPy arrays with row/column labels.

Intuition

If NumPy is the "engine" of a car, Pandas is the "dashboard" and "steering wheel". You use Pandas to organize and label data, but the engine (NumPy) does the heavy calculation.

Case 1: Interoperability

- Pandas Series and DataFrame columns are NumPy arrays with labels.
- Most NumPy functions work directly on Pandas objects.
- `df.to_numpy()` is the modern, recommended way to get the NumPy array.

Case 2: When to use which?

- Use **Pandas** for data manipulation, cleaning, and analysis (grouping, merging, time series).
- Use **NumPy** for heavy numerical computations, linear algebra, and when interfacing with ML libraries.

2. Syntax / Definition

- `df.values` : Returns the underlying NumPy array of a DataFrame.
 - `np.array(df)` : Converts a DataFrame to a NumPy array.
-

5) NumPy in ML Workflow

1. Concept Explanation

Introduction

In Machine Learning, almost every step involves NumPy:

1. **Data Preparation:** Normalizing features (scaling values between 0 and 1).
2. **Model Inputs:** ML models (like Scikit-Learn or TensorFlow) expect NumPy arrays as inputs.
3. **Weight Initialization:** Randomly starting weights for a Neural Network.
4. **Calculations:** The "Loss Function" and "Gradient Descent" are calculated using NumPy operations.

Intuition

Without NumPy, ML would be impossibly slow. Every image you see in an AI model is converted into a 3D NumPy array (Height, Width, Color Channels) before processing.

Case 1: Feature Engineering

- Creating interaction terms: `np.multiply(X[:, 0], X[:, 1])`
- Polynomial features: `X ** 2`
- Binning continuous data: `np.digitize`

Case 2: Model Predictions

- Linear Regression: `y_pred = X @ weights`
- Activation Functions: `np.sigmoid(z) = 1 / (1 + np.exp(-z))`

- Softmax: `np.exp(logits) / np.sum(np.exp(logits), axis=1, keepdims=True)`

Case 3: Performance Metrics

- Mean Squared Error: `np.mean((y_true - y_pred) ** 2)`
- Accuracy: `np.mean(y_true == y_pred)`

4. Examples (VERY IMPORTANT)

Example 1: Simple Feature Scaling (Normalization)

```
import numpy as np

# Feature: House Prices
prices = np.array([100000, 200000, 300000, 400000])

# Scale between 0 and 1: (x - min) / (max - min)
scaled_prices = (prices - np.min(prices)) / (np.max(prices) - np.min(prices))

print("Scaled prices:", scaled_prices)
```

Output

```
Scaled prices: [0.          0.33333333 0.66666667 1.          ]
```

Example 2: Building a Simple Linear Regression

```
import numpy as np

# Sample data: X (hours studied), y (test score)
X = np.array([1, 2, 3, 4, 5])
y = np.array([60, 70, 75, 85, 95])

# Add bias term
X_b = np.c_[np.ones(X.shape[0]), X]

# Calculate weights using Normal Equation: (X^T X)^-1 X^T y
weights = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y

print("Weights (bias, slope):", weights)

# Make a prediction for 6 hours
X_new = np.array([1, 6])
prediction = X_new @ weights
print("Predicted score for 6 hours:", prediction)
```

Output

```
Weights (bias, slope): [52.  8.4]
Predicted score for 6 hours: 102.4
```